

CAR BOOKING APPLICATION SECURITY REMEDIATION REPORT

=====

Course: SE4030 â Secure Software Development

Application: Car Booking Portal (PHP/MySQL)

Team: _Update with final member names and index numbers_

1. Executive Summary

A baseline security review of the legacy car booking portal uncovered multiple high-risk defects across authentication, session handling, and booking workflows. Attackers could bypass authentication via SQL injection, capture credentials hashed with obsolete MD5, hijack active sessions, and plant persistent cross-site scripting (XSS) payloads. The team refactored the application to enforce prepared statements, modern password hashing, CSRF protection, secure session cookies, robust input validation, and systematic output escaping. Google OAuth 2.0 single sign-on now supplements local accounts, reducing password surface area. The hardened build closes seven critical gaps, adds observability around database failures, and documents deployment steps so the fixes remain effective in production.

2. Application Overview

The portal allows customers to browse available vehicles, submit booking requests, and manage reservations, while an administrator back-office confirms bookings and maintains the inventory. The stack is pure PHP with a MySQL database and server-side rendered views. Authentication originally relied on custom forms without any framework support, leaving input handling and session security bespoke and error-prone.

3. Assessment Methodology

The team combined manual code review with targeted dynamic checks to enumerate vulnerabilities and confirm fixes:

- Reviewed historical commits and key entry points (login.php, register.php, carDetails.php, admin panel) to locate insecure database usage, validation gaps, and missing defenses.
- Exercised forms with Burp Suite Community Edition and browser developer tools to replicate SQL injection, CSRF, and XSS issues.
- Applied PHPStan level 1, PHP's native php -l syntax checks, and MySQL error logs to verify refactors and guardrails after each remediation.
- Re-tested exploitation attempts post-fix to ensure mitigations block the original payloads without regressing functionality.

4. Summary of Remediated Issues

ID	Vulnerability	Severity	Status
V-01	SQL injection in user and admin authentication queries	High	Fixed
V-02	Weak MD5 password hashing with no upgrade path	High	Fixed
V-03	Missing CSRF protection on authentication and booking forms	High	Fixed
V-04	Insecure session cookies and session fixation exposure	High	Fixed
V-05	Stored/reflected XSS via unescaped vehicle and booking data	High	Fixed
V-06	Booking workflow lacked server-side validation and sanitization	Medium	Fixed
V-07	Hard-coded database credentials and silent connection failures	Medium	Fixed

5. Detailed Findings

V-01: SQL Injection in Authentication Queries

- Category: Injection
- Original risk: Both login.php and admin/index.php interpolated user-supplied credentials directly into SQL strings (for example, `SELECT * FROM users WHERE email='$username' AND password='$password'`), enabling attackers to bypass authentication with crafted input or enumerate valid accounts.
- Remediation: Replaced all authentication queries with parameterised mysqli prepared statements that bind user input, eliminating string concatenation paths. The admin dashboard uses the same approach.
- Validation: Automated PHP linting succeeded, and manual replay of the payload `test@example.com' OR '1'='1` now returns the generic error message instead of logging in.

V-02: Weak Password Storage (MD5 Hashing)

- Category: Authentication
- Original risk: User and admin passwords were hashed with unsalted MD5. Attackers could crack or rainbow-table hashes rapidly, especially because the values were also used directly in SQL comparisons.
- Remediation: Registration and password upgrades now use PHP's `password_hash()` with `PASSWORD_DEFAULT`. Legacy MD5 hashes transparently upgrade to bcrypt on the next successful login to avoid locking out existing users.
- Validation: Verified that new registrations and admin credentials are stored as \$2y\$ bcrypt hashes and that legacy MD5 accounts convert after authenticating once.

V-03: Missing CSRF Protection on Key Forms

- Category: Cross-Site Request Forgery
- Original risk: Login, registration, and booking forms accepted POST requests without CSRF tokens, permitting attackers to force authenticated victims to perform sensitive actions.
- Remediation: Added a reusable helper (inc/security.php) that issues per-session CSRF tokens and validates them on submission. Tokens now guard customer, admin, and booking flows.
- Validation: Confirmed that replaying a stored POST without the token is rejected with an "Invalid session" error and no state change occurs.

V-04: Insecure Session Cookies and Session Fixation

- Category: Session Management
- Original risk: PHP sessions used default cookie parameters (no Secure, HttpOnly, or SameSite flags) and did not regenerate IDs post-login, exposing users to fixation and theft.
- Remediation: Centralized session bootstrap configures cookie flags (Secure, HttpOnly, SameSite=Strict) and regenerates the session identifier after both user and admin authentication. Logout clears the session to invalidate stolen IDs.
- Validation: Browser developer tools confirm the hardened flags, and repeated login attempts issue fresh session identifiers.

V-05: Stored and Reflected XSS via Unescaped Output

- Category: Cross-Site Scripting
- Original risk: Views such as carDetails.php, cars.php, my_account.php, and admin dashboards echoed database fields without escaping. Any HTML injected into vehicle titles, descriptions, or booking messages executed in other users' browsers.
- Remediation: Introduced an escape() helper to perform HTML encoding and applied it to every dynamic rendering path, including image paths and status labels.
- Validation: Injected <script> payloads now render harmlessly as text, and content security review confirmed no remaining raw echoes in updated templates.

V-06: Booking Workflow Validation Gaps

- Category: Input Validation
- Original risk: Booking submissions accepted arbitrary date strings and lengthy HTML in the message field. Attackers could create impossible reservations or plant malicious markup for later execution in admin views.
- Remediation: Server-side validation now enforces real calendar dates with chronological ordering, trims and strips markup from messages, and bounds length to 500 characters before

inserting records.

- Validation: Invalid dates are rejected with descriptive SweetAlert messages, and HTML tags are removed prior to persistence.

V-07: Hard-Coded Database Credentials and Silent Failures

- Category: Configuration Management
- Original risk: Database connection files hard-coded root credentials and suppressed errors, making environment-specific deployments brittle and hiding outages.
- Remediation: Connections now read DB_HOST, DB_USER, DB_PASSWORD, and DB_NAME from environment variables, enable MYSQLI_REPORT_ERROR, and abort gracefully with HTTP 500 on failure while logging the error.
- Validation: Tested by temporarily providing an invalid password and observed a controlled failure with an error log entry instead of a blank page.

6. Google OAuth 2.0 Integration

A new /oauth/google-login.php endpoint initiates the Authorization Code flow against Google's OpenID Connect provider. The callback exchanges the authorization code for tokens, validates state, and provisions or links local accounts using the users.oauth_provider and users.oauth_subject columns. Sessions are regenerated post-login, and users without local passwords are prompted to sign in exclusively via Google to reduce password sprawl. Configure the following environment variables before deploying:

- GOOGLE_CLIENT_ID
- GOOGLE_CLIENT_SECRET
- GOOGLE_REDIRECT_URI (optional; auto-derived if omitted)

7. Validation and Testing Evidence

- php -l executed on all modified PHP entry points (admin and customer flows) to ensure syntax integrity.
- Functional smoke tests covered registration, password upgrade from legacy MD5, booking submission with valid and invalid dates, and OAuth login using a Google test tenant.
- Manual review of browser developer tools confirmed secure cookie attributes and CSRF token presence on forms.

8. Residual Risks and Future Enhancements

- File uploads for vehicle images remain minimally validated; deploy-side antivirus scanning

and MIME enforcement are recommended.

- Rate limiting is absent on authentication endpoints; consider a WAF rule or application-level throttling.
- Implement centralised logging and monitoring to capture suspicious login attempts and CSRF validation failures.

9. Deployment Guidance

1. Populate .env or system environment variables for database and Google OAuth credentials before first boot.
2. Apply the updated rentcar.sql schema (includes OAuth metadata columns) or run the equivalent migration on existing databases.
3. Ensure HTTPS is enforced so Secure cookies and OAuth redirects function correctly.
4. Update README.txt with final team information and publish a demo video link prior to submission.

10. Appendix â Tooling and References

- PHP manual for password_hash, password_verify, and session configuration.
- Google Identity documentation for the OAuth 2.0 Authorization Code flow.
- OWASP ASVS v4.0.3 controls sections 2 (Authentication), 3 (Session Management), 5 (Validation), and 10 (Configuration).